

Stoquify Agent Runtime

Refined execution prompt

Act as a multidisciplinary principal review team: system architect, cyber-security architect, senior frontend engineer, UI/UX specialist, product strategist, business logic analyst, and SaaS growth advisor.

Act as a senior enterprise AI operating-system architecture team for Stoquify:

- Senior enterprise software architect: preserve Stoquify's existing module boundaries, service ownership, dependency order, tenant isolation, and integration contracts.
- Structural UI/UX design expert: propose workflow-first, role-aware, daily-use surfaces that blend into Stoquify's existing product instead of feeling like a detached AI chatbot.
- Cybersecurity and RBAC specialist: enforce tenant isolation, RBAC, module entitlement, fresh auth, maker-checker approvals, audit trails, redaction, and safe error handling.
- Business logic expert: protect traceability, approval history, source-of-truth records, evidence links, lifecycle states, and workflow correctness across POS, inventory, purchasing, reconciliation, accounting, HR, payroll, and compliance.
- Enterprise finance and controls expert: ensure every AI-assisted recommendation respects ledger integrity, reconciliation rules, close assurance, statutory boundaries, approval flows, and release gates.
- OHADA/SYSCOHADA-aware platform architect: keep statutory, tax, payroll, accounting, and country-pack rules configurable, provenance-backed, and expert-reviewable.
- SaaS modularity and growth strategist: position Stoquify as a trusted daily operating layer for African/OHADA SMBs, not merely a dashboard or generic AI wrapper.

Analyze the attached document in detail and produce a technically appropriate, professionally adapted proposal for Stoquify.

The goal is to determine how Stoquify can evolve from "a business management SaaS with dashboards" into an evidence-backed operating copilot for SMBs: a system that tells owners, accountants, managers, finance officers, HR specialists, payroll engineers, POS cashiers, and operators what happened, what matters now, what evidence supports it, what risks exist, and what controlled workflow should resolve it.

Your proposal must blend smoothly into Stoquify's current architecture and product philosophy. Do not propose a flashy AI layer that bypasses the product's existing business truth. The solution should strengthen Stoquify's moat: tenant-scoped evidence, RBAC, OHADA context, stock-to-cash-to-close continuity, workflow assurance, auditability, and safe controlled execution.

Produce a comprehensive report covering:

1. Document Interpretation

Explain the core thesis of the document, what each major point means, and why it matters strategically for Stoquify.

2. Strategic Positioning

Explain where this proposal would place Stoquify in the SMB SaaS market if executed well. Compare the resulting product category against ordinary POS, inventory, payroll, accounting, ERP, and dashboard tools.

3. Technical Architecture

Propose a shared Stoquify agent runtime instead of one agent framework per module. Define the core runtime components, including agent definitions, tool registry, skill versions, run history, evidence links, approvals, audit records, cost tracking, observability, feedback, error states, and incident handling.

4. Best-Fit Technology Stack

Recommend TypeScript-native building blocks that fit Stoquify's Next.js/TypeScript/Prisma architecture. Evaluate options such as Vercel AI SDK-style tool loops, Model Context Protocol tools, durable execution through Inngest or Trigger.dev-style systems, and observability/model-routing through Langfuse, Helicone, LiteLLM, or similar platforms. Explain what each brings, what it complicates, and whether it is worth adopting.

5. Stoquify-Native Skills

Define the domain skills Stoquify should own internally rather than outsourcing to a generic agent framework. Include skills such as trusted context resolution, permission and entitlement guarding, evidence-grounded retrieval, redaction, cash exception triage, reconciliation match suggestion, inventory risk detection, replenishment planning, close blocker navigation, payroll readiness checking, country-pack provenance, and report trust certification.

6. First Agents To Launch

Propose the first three user-facing agents:

- Command Agent
- Cash/Reconciliation Agent
- Inventory/Replenishment Agent

For each agent, define its users, daily jobs-to-be-done, read models needed, tools required, UI entry points, permissions, approval requirements, risk boundaries, success metrics, and minimum viable scope.

7. Platform Infrastructure Agents

Explain why Exception Handling and Action Orchestration should be platform infrastructure rather than visible chatbots. Define how these systems should route approvals, draft actions, preserve evidence, resume workflows, and prevent unsafe automation.

8. Safety And Authority Model

Define what agents must never do directly:

- no direct Prisma writes
- no direct ledger posting
- no statutory filing
- no role or permission changes
- no payroll approval
- no close-pack certification
- no cash adjustment
- no stock write-off

Explain the controlled service-action model agents must use instead: recommendation, draft, approval, fresh auth, idempotency, service-owned execution, audit logging, and rollback/exception handling.

9. UI/UX Integration

Propose how the agent experience should appear inside Stoquify without disrupting the product. Focus on role-aware command surfaces, module-level assistant panels, exception queues, evidence drawers, action drafts, approval timelines, notification patterns, and daily operating summaries. Avoid generic "chat everywhere" design.

10. Implementation Roadmap

Create a phased plan:

- First 30 days: runtime schema, tool wrappers, read-only Command Agent, trusted context, permission guard, evidence retrieval, redaction, freshness checks.
- Days 31-60: Cash/Reconciliation Agent, Inventory/Replenishment Agent, read-and-draft mode, approval workflows, feedback loops.
- Days 61-90: low-risk controlled service actions, idempotent execution, tracing, cost tracking, agent assurance monitoring.
- After 90 days: Purchasing/AP Agent, Close and Compliance Agent, Payroll Readiness Agent, Customer Success/Adoption Agent.

11. Least-Code Highest-Leverage Strategy

Identify how Stoquify can achieve the most professional result with the least code by reusing existing services, read models, RBAC checks, audit logs, workflow states, and evidence trails instead of building a parallel AI system.

12. Risks And Tradeoffs

List the architectural, security, compliance, product, cost, operational, and user-trust risks. For each risk, propose mitigation.

13. Success Criteria

Define measurable success criteria: time saved, exception resolution rate, reconciliation speed, inventory stockout reduction, accepted recommendation rate, fewer stale dashboards, reduced close blockers, lower support burden, agent error rate, audit completeness, and role-specific daily engagement.

14. Final Recommendation

Conclude whether Stoquify should pursue this proposal, under what constraints, and what the first implementation slice should be.

Non-goals:

- Do not propose replacing Stoquify's existing business services with AI.
- Do not propose one independent agent framework per module.
- Do not propose direct agent writes to financial, statutory, payroll, permission, or inventory-control records.
- Do not propose a generic chatbot as the main product strategy.
- Do not ignore OHADA/SYSCOHADA, RBAC, tenant isolation, evidence, auditability, or workflow control.

The final output should be a professional, structured report suitable for product, engineering, security, and executive review.